

Current Project Shape

Human Firewall is a security awareness and threat response platform with these main parts:

- Flask backend in backend/app.py with SQLite data access in backend/database.py
- Next.js dashboard in dashboard/src/app
- Employee portal with behavior score, activity log, gamification, and “Spot the Fake”
- Admin SOC dashboard with incidents, behavior analytics, policy decisions, leaderboard, compliance widgets, GoPhish controls, and mock webmail
- GoPhish integration for campaign sync/launch
- n8n workflows for phishing simulation and Telegram-based threat reporting
- Current persistence is split: important backend state is SQLite, while several dashboard surfaces still use in-memory demo data in dashboard/src/lib/seed.ts

10 Enterprise-Grade Feature Proposals

1. Unified Enterprise Data Layer

Move incidents, behavior scores, policy decisions, threat cache, AI summaries, and audit data out of Next.js in-memory store into the backend database. This is the foundation for production reliability, reporting, and multi-user consistency.

2. Role-Based Access Control

Add proper roles such as SOC Analyst, SOC Manager, Campaign Admin, Compliance Officer, and Employee. Current auth is mostly admin password, bearer secret, and employee dashboard tokens; enterprise use needs scoped permissions.

3. Full Audit Trail

Record every sensitive action: campaign launch, incident status change, login, report submission, user score adjustment, policy decision, and admin configuration change. Add searchable audit views for compliance and incident review.

4. Incident Case Management

Upgrade incidents from simple open/closed tickets into full cases with assignee, SLA, comments, evidence attachments, status workflow, severity changes, related reports, and escalation history.

5. Adaptive Training Assignment Engine

Automatically assign training modules based on behavior score, click history, credential submission, division risk, and report quality. The current “Spot the Fake” can become one module in a broader remediation engine.

6. Campaign Template Governance

Add a managed library for phishing templates with approval workflow, risk level, target audience, localization, legal/compliance notes, and reuse history before campaigns are launched through GoPhish.

7. Threat Intelligence Enrichment Pipeline

Normalize VirusTotal, urlscan.io, file hash, domain age, WHOIS, screenshot, redirect chain, and sandbox verdicts into a single threat object. Use that to drive scoring, policy decisions, and SOC recommendations.

8. Executive Risk & Compliance Reporting

Generate monthly board-ready reports: division risk trends, campaign effectiveness, reporting rate, time-to-report, training completion,

repeat offenders, incident MTTR, and compliance posture for UU PDP/BSSN-style controls.

9. Enterprise User & Org Sync

Integrate employee identity with HRIS/LDAP/Azure AD/Google Workspace style sources. Support departments, managers, employment status, groups, and automatic target segmentation for campaigns and reporting.

10. Adaptive Secure Gateway API

Productize the existing policy-decision concept into a real enforcement API: given user, URL/file, device context, and threat score, return allow/warn/block/quarantine/notify. This would turn the platform from awareness-only into active human-risk-based protection.

My recommended build order would be: unified data layer, RBAC, audit trail, incident case management, then adaptive training. Those unlock the rest without building more demo-only surface area.

It's a strong hackathon PoC because it has a clear story: phishing simulation, teachable moment, Telegram reporting, automated triage, SOC dashboard, and employee behavior scoring. That is much better than a static "security dashboard" demo because it shows an end-to-end human firewall loop. The biggest strength is the workflow: user clicks or reports something, automation reacts, risk score changes, SOC sees it, and training follows. That maps well to a real enterprise pain point. The main weakness is product polish versus architecture maturity. Some dashboard data is still in-memory demo state while backend data lives in SQLite. For a hackathon PoC, that is acceptable, but you should be transparent: position it as an integrated prototype proving the workflow, not a production-ready platform.

For submission, I'd frame it around these differentiators:

1. Closed-loop security awareness

Not just phishing training, but detection, feedback, scoring, escalation, and retraining.

2. Human-risk-based response

Different users get different actions based on behavior score, not one-size-fits-all blocking.

3. SOC + employee dual experience

The project has both admin command center and personal employee portal.

4. Automation-first design

n8n, GoPhish, Telegram, VirusTotal/urlscan-style enrichment, and incident creation make it feel operational.

5. Localized enterprise angle

Indonesian language, UU PDP/BSSN-style compliance widgets, and division-level risk reporting make it more grounded than generic security demos.

Before submitting, I would prioritize only a few things:

- Fix visible encoding issues like đŸŠ, Â©, etc. They hurt perceived quality immediately.
- Make sure the demo flow is reliable from start to finish.
- Prepare a 3-5 minute scripted demo with one employee click, one Telegram report, one SOC incident, and one behavior-score change.
- Clearly label mock/demo components, especially in-memory dashboard data.
- Avoid showing the malicious-pdf tool unless it is directly relevant and safe to explain. It may distract judges or raise unnecessary concerns.

My honest take: this is very viable for a hackathon PoC. It already has enough surface area. Don't add many new features before submission; stabilize the story, polish the UI text/encoding, and make the demo flow feel intentional.

I analyzed the repo read-only. No code was modified.

Frontend Architecture

The frontend is a Next.js app under `dashboard/src`, using the App Router, React client pages, local CSS, lucide-react, and recharts.

Main UI surfaces:

- Employee dashboard: `dashboard/src/app/page.tsx`

Shows personal behavior score, rank, streak, activity timeline, reporting badges, and the “Spot the Fake” training game.

- Admin/SOC dashboard: `dashboard/src/app/admin/page.tsx`

Shows overview metrics, threat cache, AI summaries, behavior scores, policy decisions, leaderboard, GoPhish controls, and mock webmail.

- Employee auth page: `dashboard/src/app/auth/page.tsx`

Validates Telegram magic-link tokens.

- Admin login page: `dashboard/src/app/admin/login/page.tsx`

The frontend has two data modes:

- Real Flask-backed routes via Next.js proxy routes, for auth, user activity, eligibility, GoPhish, compliance, leaderboard, reporting summary, and event submission.

- In-memory demo routes via `dashboard/src/lib/store.ts` and `dashboard/src/lib/seed.ts`, for incidents, cache, AI summaries, policy decisions, and behavior demo data.

Architecturally, Next.js currently acts as both presentation layer and lightweight API gateway.

Backend Architecture

The backend is Flask, not FastAPI, located in backend.

Main entrypoint:

- backend/app.py

Initializes Flask, CORS, SQLite schema, route blueprints, global auth guard, and /health.

Route modules:

- backend/routes/auth.py: admin auth, token registration, Telegram user lookup.

- backend/routes/events.py: phishing redirect, event capture, OTP, user profile, eligibility, activity.

- backend/routes/incidents.py: incident ticket create/list/update.

- backend/routes/admin_api.py: dashboard summary, leaderboard, compliance, GoPhish control.

- backend/gamification_routes.py: threat reports, report summary, daily quiz completion.

Business/data logic is centralized in backend/database.py. That file owns schema initialization, migrations, event recording, scoring, reporting, OTP, incidents, dashboard summaries, and token validation.

Deployment is Docker Compose based:

- n8n: orchestration and Telegram/VirusTotal/urlscan workflows.
- gophish: phishing campaign engine.
- flask_api: backend and SQLite persistence.
- dashboard: Next.js UI/API gateway.

See docker-compose.yml.

Authentication Flow

There are three auth paths.

Admin auth:

- Admin enters password in the Next.js admin login page.
- Next.js calls /api/auth/admin-login.
- That proxies to Flask /api/auth/admin.
- Flask compares the submitted password with ADMIN_PASSWORD.
- Frontend stores returned admin user object in localStorage via dashboard/src/context/AuthContext.tsx.

Employee dashboard auth:

- Telegram/n8n requests or creates a magic link.
- Next.js /api/auth/magic-link creates a temporary token in the in-memory store.
- It also attempts to register the token into Flask /api/auth/register-token.
- User opens /auth?token=....

- Next.js validates token through its in-memory store and stores the employee user in localStorage.
- Later employee-specific Flask calls pass email + token; Flask validates token ownership through dashboard_tokens.

Telegram onboarding / OTP auth:

- User starts via Telegram.
- n8n/Telegram flow calls Flask OTP endpoints.
- /api/otp/create stores OTP and writes mock email.
- /api/otp/verify maps telegram_chat_id to employee email.
- user_history.telegram_chat_id becomes the identity bridge between Telegram and employee record.

Important weakness: frontend session state is stored in localStorage, there is no secure HTTP-only session cookie, and admin auth is effectively client-side persisted after password verification.

API Endpoints

Backend Flask endpoints:

- GET /health
- GET /
- GET/POST /admin/login
- GET /admin/logout
- POST /api/auth/admin
- POST /api/auth/register-token
- GET /api/telegram/user
- GET /redirect-handler
- POST /api/fake-login-submit

- GET /api/user-history
- POST /api/event
- POST /api/register-telegram
- POST /api/otp/create
- POST /api/otp/verify
- GET /api/emails
- GET /api/user-profile
- GET /api/user-eligibility
- GET /api/user-activity
- POST /api/incidents
- GET /api/incidents
- PATCH /api/incidents/<ticket_id>
- GET /api/dashboard-summary
- GET /api/leaderboard
- GET /api/compliance-summary
- GET /api/admin/gophish/campaigns
- GET /api/admin/gophish/resources
- POST /api/admin/gophish/sync
- POST /api/admin/gophish/launch
- POST /api/reports
- GET /api/employee/<employee_id>/reports-summary
- POST /api/quiz/complete

Next.js API routes:

- Real proxy routes: admin login, admin compliance, leaderboard, GoPhish, emails, event, user activity, user eligibility, employee reports summary.
- Demo/in-memory routes: /api/incident, /api/cache, /api/summary, /api/behavior, /api/policy, plus seeded login history.

Database Structure

SQLite schema is initialized in backend/database.py.

Core tables:

- user_history

Employee aggregate state: email, division, click count, training counts, Telegram chat ID, points, badge, report counters, daily streak, last quiz date.

- events

Raw activity log: email, division, event type, assigned tier, campaign ID, timestamp.

- incidents

SOC tickets from simulations or real-world reports: ticket ID, source type, reported URL, division, severity, VirusTotal verdict, urlscan verdict, screenshot URL, checklist, file hash, original filename, status, timestamps.

- registration_otp

Temporary OTP records for Telegram onboarding.

- inbox_emails

Mock webmail inbox records.

- link_tokens

Short-lived Telegram deep-link tokens.

- dashboard_tokens

Longer-lived employee dashboard tokens.

- threat_reports

Submitted user reports after triage, including verdict, target, source engine, dedupe status, raw scores, and gamification flag.

- daily_events

Per-user daily events, currently used for quiz completion/streak tracking.

This schema is good for a PoC, but enterprise deployment would need stronger migrations, relational constraints, audit tables, and likely PostgreSQL.

Gamification System

Gamification is split into points, badges, report rewards, and daily streaks.

Point model:

- Starts users at 100 points.
- Range is clamped from 0 to 200.
- clicked_link: -10
- submitted_data: -20
- confirmed malicious/suspicious report: +15
- correct "Spot the Fake": +5
- daily quiz completion: +10

Badge model:

- Behavioral badge from points:

Sentinel for high points, Guardian for normal range, Vulnerable for low points.

- Reporting badges from backend/badges.json:
thresholds at 1, 3, 5, and 10 confirmed malicious/suspicious reports.

Training/game model:

- Employee dashboard checks eligibility.
- Safe users may be locked out of required training.
- Users can be limited by 24-hour cooldown.
- Daily quiz completion updates daily_streak server-side.

Dedupe logic:

- Threat reports dedupe on (email, target) if prior verdict was malicious/suspicious.
- Duplicate or clean reports increment total reports but do not count toward malicious report badges.

Reporting System

There are two reporting loops.

Simulation reporting:

- GoPhish sends phishing link.
- User click hits Flask /redirect-handler.
- Flask records clicked_link, assigns tier, updates points, and notifies n8n.

- Tier 1 gets educational page; repeat/chronic users can get stronger intervention.

- Fake credential submission hits /api/fake-login-submit, records submitted_data, applies larger point penalty, and notifies n8n/SOC.

Real-world threat reporting:

- Employee reports URL/file via Telegram bot.
- n8n performs triage using external tools such as VirusTotal/urlscan.
- n8n can create incident tickets via /api/incidents.
- n8n can record gamified threat reports via /api/reports.
- SOC dashboard displays incident/reporting-related state, though some SOC panels are still demo/in-memory.

Missing Enterprise Cybersecurity Features

Most important gaps:

- Proper RBAC/ABAC for admin roles, SOC roles, managers, compliance officers, and employees.
- Secure session management with HTTP-only cookies, refresh/session expiry, CSRF protection, and logout invalidation.
- Centralized audit logging for every admin action, campaign launch, incident update, score change, and auth event.
- Production database layer: PostgreSQL, migrations, constraints, backups, retention, and multi-instance-safe storage.
- Real incident case management: assignee, SLA, comments, evidence,

escalation, severity history, and closure reason.

- SIEM/SOAR integrations: Splunk, Sentinel, QRadar, TheHive, Jira, ServiceNow, Slack/Teams.
- Enterprise identity sync: Azure AD/Entra ID, Google Workspace, LDAP/SCIM, org chart, manager mapping.
- Policy/risk engine persisted in backend rather than demo-only Next.js memory.
- Data privacy controls: PII minimization, encryption at rest, field-level access, retention policy, export/delete workflows.
- Security hardening: rate limits, request signing for n8n/webhooks, secret rotation, input validation, SSRF-safe URL scanning boundaries.
- Campaign governance: approval workflow, template review, target segmentation, quiet hours, frequency caps.
- Detection enrichment: domain age, WHOIS, SPF/DKIM/DMARC, attachment sandboxing, redirect-chain analysis, YARA/Sigma-style rules.
- Reporting quality scoring: reward accurate reports, suppress spammy duplicate reports, track false positives.
- Compliance evidence packs: downloadable audit reports mapped to UU PDP, ISO 27001, NIST CSF, SOC 2 controls.
- Multi-tenant support if intended for multiple customers/business units.

Overall: as a hackathon PoC, this is a strong end-to-end concept. The biggest architectural issue is the split between real Flask/SQLite state and demo-only Next.js in-memory state. The highest-value next step is to unify persistence and auth before adding more features.

Competition Review

Category	Score	Judge Notes
Innovation	8/10	Strong concept: phishing simulation, Telegram reporting, automated triage, behavior scoring, gamification, and SOC dashboard in one loop. It is more complete than a typical awareness demo. Not fully novel, but the integrated “human-risk adaptive response” angle is compelling.
Technical Architecture	6.5/10	Good service separation: Next.js, Flask, SQLite, n8n, GoPhish, Telegram, external scanning APIs. The weakness is inconsistent state ownership: some critical dashboard data is real backend state, while other parts are in-memory demo data inside Next.js.
Cybersecurity Realism	7/10	Realistic workflows: phishing click tracking, credential submission

simulation, VirusTotal/urlscan-style assessment, incident creation, SOC escalation, training intervention. Realism drops because auth, auditability, RBAC, webhook security, and evidence handling are still PoC-level.

UI/UX	7.5/10	The product has both employee and SOC experiences, which is a big plus. The dashboard feels alive and demo-friendly. Main issue: visible encoding glitches like "Š", some dense panels, and mixed demo/real data can reduce polish.
-------	--------	---

Scalability	4.5/10	Dockerized architecture helps, but SQLite, in-memory Next.js store, localStorage auth, and polling-heavy UI limit production scalability. It works for demo scale, not enterprise scale.
-------------	--------	--

Enterprise Readiness	4/10	The concept fits enterprise, but implementation lacks RBAC, audit logs, identity provider integration, SIEM/SOAR integration, case management, secure sessions, policy governance, retention controls, and production-grade database/migrations.
----------------------	------	--

Overall Score: 6.5/10

As a hackathon PoC, I would rate it higher, around 8/10, because it tells a strong story and demonstrates an end-to-end workflow. As a cybersecurity startup competition submission, I'd score it lower because judges will look for production credibility, defensibility, security controls, and enterprise buyer readiness.

What Would Raise It To 10/10

To reach 10/10, the biggest improvement is not adding more flashy features. It is making the platform feel like a serious cybersecurity product.

1. Unify the Data Layer

Move demo-only Next.js in-memory data into the backend database. Incidents, behavior scores, policy decisions, threat cache, summaries, and reporting metrics should all have one source of truth.

2. Add Enterprise Authentication

Replace localStorage auth with secure HTTP-only sessions or proper token handling. Add RBAC roles such as SOC Analyst, SOC Lead, Campaign Admin, Compliance Officer, Manager, and Employee.

3. Add Audit Logging

Every sensitive action should be logged: campaign launch, incident update, score change, admin login, report verdict, training completion, and GoPhish sync.

4. Turn Incidents Into Case Management

Add assignee, SLA, comments, severity history, evidence, related reports, escalation status, closure reason, and timeline.

5. Harden Webhooks And APIs

Add request signing, replay protection, rate limiting, stricter validation, secret rotation, and clear trust boundaries between n8n, Flask, Next.js, and GoPhish.

6. Improve Enterprise Integrations

Add at least mock-ready connectors for Microsoft Entra ID/Google Workspace, SIEM/SOAR, ServiceNow/Jira, Slack/Teams, and email security gateways.

7. Make The Risk Engine Explainable

Show why a user/division score changed, what evidence contributed, and what policy action was triggered. Judges like explainability in security products.

8. Polish The Demo

Fix encoding issues, remove unclear mock data, make the click/report/SOC/training flow deterministic, and prepare a tight 3-5 minute storyline.

If I were judging, I'd see this as a promising cybersecurity workflow platform with strong demo potential. To win against startup-grade competitors, it needs stronger production architecture, security governance, and enterprise trust controls.

CTO Assessment

This repository is a strong hackathon-grade prototype with a compelling product narrative: employees encounter phishing simulations, receive teachable feedback, report suspicious threats through Telegram, automated workflows enrich the threat, SOC sees incidents, and the employee risk score changes over time.

The product thesis is good. The architecture is not yet competition-grade or investor-grade. The main job is to turn a clever integrated demo into a credible security platform.

1. Architecture Review

Current architecture:

- Frontend: Next.js dashboard with employee portal, SOC admin dashboard, magic-link auth, gamification UI, GoPhish controls, and demo telemetry.
- Backend: Flask API with SQLite persistence, route blueprints, event tracking, incidents, OTP, gamification, compliance summaries, GoPhish integration.
- Automation: n8n orchestrates Telegram bot flows, VirusTotal/urlscan-style enrichment, escalation, and backend calls.
- Simulation: GoPhish handles phishing campaigns.
- Persistence: Mixed model. Some data is real SQLite-backed backend state; some dashboard data is in-memory Next.js demo state.
- Auth: Admin password auth, employee Telegram magic-link flow, dashboard

tokens, localStorage frontend session.

- Product surface: SOC dashboard, employee dashboard, training game, leaderboard, compliance widgets, threat reporting, mock webmail.

The product is architecturally coherent as a PoC, but not yet as a production cybersecurity platform.

2. What Should Change First

First priority: unify the source of truth.

Right now, the product has two realities:

- Real backend state in Flask/SQLite.
- Demo state in Next.js memory.

That creates credibility risk. Judges, customers, and investors will notice if dashboard numbers are not tied to real workflows.

The first major change should be:

- Move incidents, threat cache, behavior scores, policy decisions, AI summaries, login history, and reporting metrics into the backend data layer.
- Replace demo-only API routes with backend-backed routes.
- Define clear ownership: Next.js renders, Flask/API owns business logic and persistence, n8n orchestrates workflows.

Until this is fixed, adding more features increases confusion.

3. Prioritized Roadmap

Milestone 1: Product Stabilization & Demo Integrity

Effort: 1-2 weeks

- Fix encoding issues across UI/docs.
- Remove or clearly label demo-only data.
- Make one deterministic end-to-end demo path: phishing click → training → score change → SOC event → report → incident.
- Add seed/reset scripts for competition demos.
- Document architecture and demo script.

Goal: make the product presentation reliable.

Milestone 2: Unified Backend Data Model

Effort: 2-4 weeks

- Move in-memory Next.js state into backend persistence.
- Add real tables for threat cache, behavior scores, policy decisions, summaries, admin login events, and audit logs.
- Replace Next.js mock APIs with Flask-backed APIs.
- Add basic migration management.

Goal: make all dashboards reflect real system state.

Milestone 3: Secure Authentication & RBAC

Effort: 2-3 weeks

- Replace localStorage-only auth with secure sessions or properly scoped tokens.
- Add roles: Employee, SOC Analyst, SOC Lead, Campaign Admin, Compliance Officer, Super Admin.
- Protect admin APIs based on role.
- Add session expiry, logout invalidation, and token revocation.

Goal: stop looking like a demo app and start looking enterprise-ready.

Milestone 4: Incident Case Management

Effort: 3-5 weeks

- Add assignee, SLA, severity history, comments, evidence, related reports, status transitions, and closure reason.
- Add case timeline.
- Add escalation workflow.
- Add exportable case report.

Goal: make SOC users believe they can operate from the platform.

Milestone 5: Risk Engine V1

Effort: 3-5 weeks

- Formalize user, division, campaign, and threat scoring.
- Make score changes explainable.
- Track signal provenance: click, report, training, false positive, credential submission, repeat behavior.

- Add adaptive policy output: allow, warn, block, retrain, escalate.

Goal: make the product defensible as “human-risk adaptive security,” not just awareness training.

Milestone 6: Enterprise Integrations

Effort: 4-8 weeks

- Add Microsoft Entra ID / Google Workspace mock or real integration.
- Add SIEM/SOAR export: Splunk, Sentinel, QRadar, TheHive, or generic webhook.
- Add Jira/ServiceNow ticket sync.
- Add Slack/Teams notification channel.
- Add email security gateway ingest/export concept.

Goal: show enterprise buying fit.

Milestone 7: Compliance & Executive Reporting

Effort: 2-4 weeks

- Add board-level reports.
- Map metrics to ISO 27001, NIST CSF, SOC 2, UU PDP/BSSN-style controls.
- Add PDF/CSV export.
- Add monthly trend analytics.

Goal: give CISOs a business reason to care.

Milestone 8: Production Hardening

Effort: 4-8 weeks

- Move from SQLite to PostgreSQL.
- Add API validation, rate limiting, request signing, replay protection, structured logging.
- Add secrets management strategy.
- Add tests for scoring, auth, incidents, and workflows.
- Add container health checks and deployment documentation.

Goal: prove this can survive outside a laptop.

4. Technical Debt

Highest-impact technical debt:

- Mixed real/demo data stores.
- Next.js API routes acting as both proxy and fake backend.
- localStorage session persistence.
- Weak admin auth model.
- SQLite with ad hoc migrations.
- No audit log.
- No formal API schema.
- No typed backend contracts.
- Polling-heavy UI.
- Encoding corruption in text/assets.
- Business logic concentrated in a very large database.py.
- Security-sensitive integrations rely on shared secrets without robust signing/replay controls.
- No test coverage for critical flows except limited manual gamification testing.

- GoPhish API integration disables TLS verification because of self-signed certs.
- n8n workflows are powerful but not versioned/tested like application code.

5. Architectural Risks

Major risks:

- Trust boundary confusion: Next.js, Flask, n8n, Telegram, and GoPhish all exchange sensitive actions without a strong integration security model.
- Demo drift: Dashboard may show values not caused by real events.
- State loss: In-memory dashboard data disappears on restart.
- Auth bypass risk: Frontend-stored session data can be manipulated.
- Scaling risk: SQLite and polling are fine for a PoC, weak for multi-user enterprise load.
- Compliance risk: Employee behavior data is sensitive and needs retention, privacy, and access controls.
- Operational risk: n8n workflows are essential but external to the typed/tested codebase.
- Security optics risk: The malicious-pdf folder may distract judges unless carefully framed or removed from the product demo.

6. Strongest Selling Points

The strongest product assets are:

- Closed-loop human firewall workflow.
- Employee and SOC views in one platform.
- Real phishing simulation integration through GoPhish.
- Telegram-based user reporting and onboarding.
- Behavior scoring tied to actual user actions.
- Gamified reporting and training.
- Adaptive training concept.
- Compliance and executive dashboard angle.
- Localized Indonesian enterprise context.
- Clear demo story that non-technical judges can understand quickly.

The product is not just “training.” It is awareness, detection, response, and behavior analytics in one loop.

7. What Makes It Different

Most competitors sit in one category:

- Phishing simulation platforms.
- Security awareness training.
- SOAR tools.
- SIEM dashboards.
- User behavior analytics.
- Email reporting plugins.

This product combines those into a human-risk operating layer.

The differentiator should be stated as:

> Human Firewall turns employee security behavior into a real-time

> adaptive defense signal.

That means:

- Risky users get stronger interventions.
- Reliable reporters become sensors.
- SOC gets human-context-rich incidents.
- Training is triggered by behavior, not calendar schedules.
- Division risk becomes measurable and actionable.

That positioning is stronger than “phishing awareness platform.”

8. What NOT To Build

Do not build these yet:

- A full LMS.
- Too many mini-games.
- A custom SIEM.
- A custom phishing engine to replace GoPhish.
- A custom workflow engine to replace n8n.
- Complex AI chatbot features.
- Browser extension enforcement before the backend is credible.
- Multi-tenant SaaS before RBAC, audit, and data model are fixed.
- More decorative dashboards.
- More mock features that are not connected to real workflows.
- Offensive tooling demos such as malicious PDF generation unless the competition explicitly rewards red-team tooling.

The product already has enough surface area. The next work should deepen credibility, not widen the demo.

9. Investor Credibility

Investors will believe this has commercial potential if you can prove five things.

First, prove the buyer pain: phishing remains expensive, training completion does not equal behavior change, and SOC teams lack human-context signals.

Second, prove the wedge: start with phishing simulation plus employee reporting plus SOC dashboard. That is understandable and sellable.

Third, prove expansion: grow into adaptive access policy, identity risk, compliance reporting, and security culture analytics.

Fourth, prove enterprise readiness: RBAC, audit logs, SSO, SIEM/SOAR integration, case management, privacy controls, and production deployment.

Fifth, prove measurable ROI:

- Reduced phishing click rate.
- Increased reporting rate.
- Faster time-to-report.
- Lower SOC triage time.
- Fewer repeat offenders.
- Improved division risk score.
- Evidence for compliance audits.

The strongest investor story is:

- > Companies already spend on awareness training and SOC tooling, but those
- > systems are disconnected. Human Firewall connects employee behavior,
- > threat reporting, automated triage, and adaptive response into one
- > measurable security layer.

That is a commercial thesis worth pursuing. The current repo is a promising prototype. To win internationally, it needs disciplined product hardening, not feature sprawl.

Below is a structured engineering backlog aimed at turning this repository from a strong hackathon PoC into a competition-ready cybersecurity product.

Legend:

- CR: Critical for competition readiness
- GFI: Good first issue
- Post-MVP: Can wait until after MVP

Epic 1: Demo Integrity & Product Polish

Feature	Text/Encoding Cleanup
Task	Fix corrupted UI strings like đŸš”, Â©, mojibake across frontend/backend templates
Difficulty	Easy
Est. Hours	6
Dependencies	None
Files Likely Affected	dashboard/src/app/page.tsx, dashboard/src/app/ admin/page.tsx, backend/templates, README.md
Tags	CR, GFI

Feature	Text/Encoding Cleanup
Task	Normalize copy language: choose Indonesian or English per screen and make labels consistent
Difficulty	Easy
Est. Hours	4
Dependencies	Encoding cleanup
Files Likely Affected	Same as above
Tags	CR, GFI

Feature	Deterministic Demo Flow
Task	Create a documented 5-minute demo path: click simulation, training, Telegram report, SOC incident, score change
Difficulty	Medium
Est. Hours	5
Dependencies	None
Files Likely Affected	README.md, DEVELOPER_GUIDE.md
Tags	CR

Feature	Deterministic Demo Flow
Task	Add seed/reset instructions for predictable demo data
Difficulty	Medium
Est. Hours	6
Dependencies	Understand seed flow
Files Likely Affected	backend/seed_data.py, dashboard/src/lib/seed.ts, docs
Tags	CR

Feature	Demo Safety
Task	Decide whether to remove, hide, or clearly isolate

malicious-pdf from product demo narrative

Difficulty Easy
Est. Hours 2
Dependencies None
Files Likely Affected malicious-pdf, docs
Tags CR

Epic 2: Unified Data Layer

Feature Backend Source of Truth
Task Add backend tables for threat cache, policy
 decisions, AI summaries, and admin login events
Difficulty Medium
Est. Hours 10
Dependencies Schema decision
Files Likely Affected backend/database.py
Tags CR

Feature Backend Source of Truth
Task Move incident dashboard data from Next.js in-
 memory store to Flask-backed /api/incidents
Difficulty Medium
Est. Hours 8
Dependencies Backend table/API alignment
Files Likely Affected dashboard/src/app/api/incident/route.ts, backend/
 routes/incidents.py
Tags CR

Feature Backend Source of Truth
Task Replace /api/cache in-memory route with backend-
 backed threat cache endpoint

Difficulty	Medium
Est. Hours	8
Dependencies	Threat cache table
Files Likely Affected	dashboard/src/app/api/cache/route.ts, new Flask route
Tags	CR

Feature	Backend Source of Truth
Task	Replace /api/policy in-memory route with backend-backed policy decision endpoint
Difficulty	Medium
Est. Hours	8
Dependencies	Policy table
Files Likely Affected	dashboard/src/app/api/policy/route.ts, new Flask route
Tags	CR

Feature	Backend Source of Truth
Task	Replace /api/summary in-memory route with backend-backed summary endpoint
Difficulty	Medium
Est. Hours	8
Dependencies	Summary table
Files Likely Affected	dashboard/src/app/api/summary/route.ts, new Flask route
Tags	CR

Feature	Backend Source of Truth
Task	Replace demo behavior score store with backend-calculated behavior scores
Difficulty	Hard

Est. Hours 16

Dependencies Scoring model

Files Likely Affected dashboard/src/app/api/behavior/route.ts, backend/
 database.py

Tags CR

Feature Data Cleanup

Task Remove or demote dashboard/src/lib/store.ts to
 demo-only utility

Difficulty Medium

Est. Hours 4

Dependencies All replacement routes done

Files Likely Affected dashboard/src/lib/store.ts, dashboard/src/lib/
 seed.ts

Tags CR

Epic 3: Authentication & Authorization

Feature Session Hardening

Task Replace frontend-only localStorage auth with HTTP-
 only session cookie or secure token strategy

Difficulty Hard

Est. Hours 20

Dependencies Auth design

Files Likely Affected dashboard/src/context/AuthContext.tsx, auth
 routes, Flask auth

Tags CR

Feature Admin Auth

Task Add server-side admin session validation for all
 admin Next.js routes

Difficulty	Medium
Est. Hours	10
Dependencies	Session hardening
Files Likely Affected	dashboard/src/app/api/admin, backend/routes/auth.py
Tags	CR

Feature	RBAC
Task	Define roles: Employee, SOC Analyst, SOC Lead, Campaign Admin, Compliance Officer, Super Admin
Difficulty	Medium
Est. Hours	4
Dependencies	None
Files Likely Affected	Docs, backend auth model
Tags	CR

Feature	RBAC
Task	Add role field and role checks to protected backend routes
Difficulty	Hard
Est. Hours	16
Dependencies	Role model
Files Likely Affected	backend/app.py, backend/routes
Tags	CR

Feature	Token Security
Task	Add token revocation and expiry handling for dashboard tokens
Difficulty	Medium
Est. Hours	8
Dependencies	Dashboard token model

Files Likely Affected backend/database.py, backend/routes/auth.py

Tags CR

Feature Demo Auth Cleanup

Task Remove hardcoded demo-magic-link-2026 bypass or
isolate behind demo mode flag

Difficulty Easy

Est. Hours 3

Dependencies None

Files Likely Affected backend/database.py, dashboard/src/lib/seed.ts

Tags CR, GFI

Epic 4: API Security & Integration Hardening

Feature Webhook Security

Task Add HMAC request signing for n8n-to-Flask and n8n-
to-Next.js calls

Difficulty Hard

Est. Hours 16

Dependencies Secret strategy

Files Likely Affected backend/app.py, backend/routes, n8n workflows

Tags CR

Feature Webhook Security

Task Add replay protection using timestamp + nonce

Difficulty Hard

Est. Hours 12

Dependencies HMAC signing

Files Likely Affected Backend middleware, DB nonce table

Tags CR

Feature	Input Validation
Task	Add strict validation for incident, event, report, campaign launch payloads
Difficulty	Medium
Est. Hours	12
Dependencies	None
Files Likely Affected	backend/routes, backend/gamification_routes.py
Tags	CR

Feature	Rate Limiting
Task	Add basic rate limits for auth, OTP, reports, and public event endpoints
Difficulty	Medium
Est. Hours	8
Dependencies	None
Files Likely Affected	backend/app.py, requirements
Tags	CR

Feature	CORS/Headers
Task	Add secure headers and tighten CORS behavior
Difficulty	Easy
Est. Hours	4
Dependencies	None
Files Likely Affected	backend/app.py, Next config
Tags	CR, GFI

Feature	GoPhish Security
Task	Replace blanket verify=False with configurable TLS validation for production mode
Difficulty	Medium
Est. Hours	5

Dependencies Cert strategy

Files Likely Affected backend/gophish_client.py

Tags Post-MVP

Epic 5: Incident Case Management

Feature Case Model

Task Extend incidents with assignee, SLA, priority,
closure reason, status history

Difficulty Hard

Est. Hours 16

Dependencies Unified backend data

Files Likely Affected backend/database.py, backend/routes/incidents.py

Tags CR

Feature Case Timeline

Task Add incident timeline endpoint showing creation,
updates, comments, evidence

Difficulty Hard

Est. Hours 14

Dependencies Case model

Files Likely Affected Backend incidents route, dashboard admin page

Tags CR

Feature Analyst Workflow

Task Add assign/reassign, escalate, resolve, reopen
actions

Difficulty Medium

Est. Hours 12

Dependencies Case model

Files Likely Affected dashboard/src/app/admin/page.tsx, incidents API

Tags CR

Feature Comments

Task Add analyst comments to incidents

Difficulty Medium

Est. Hours 8

Dependencies Case model

Files Likely Affected Backend incident tables/routes, admin UI

Tags CR

Feature Evidence

Task Store URL scan screenshot, file hash, raw verdict,
checklist as structured evidence

Difficulty Medium

Est. Hours 10

Dependencies Reporting system

Files Likely Affected Backend DB/routes, admin UI

Tags CR

Feature Export

Task Export incident case as JSON/PDF-ready report

Difficulty Medium

Est. Hours 8

Dependencies Case timeline

Files Likely Affected Backend export route

Tags Post-MVP

Epic 6: Gamification & Behavior Risk Engine

Feature Score Explainability

Task Add score-change ledger table for every point

adjustment

Difficulty Hard

Est. Hours 14

Dependencies Unified DB

Files Likely Affected backend/database.py

Tags CR

Feature Score Explainability

Task Show “why your score changed” in employee activity
 timeline

Difficulty Medium

Est. Hours 8

Dependencies Score ledger

Files Likely Affected dashboard/src/app/page.tsx, user activity API

Tags CR

Feature Behavior Model

Task Formalize risk score formula separately from raw
 points

Difficulty Hard

Est. Hours 14

Dependencies Score ledger

Files Likely Affected Backend scoring module

Tags CR

Feature Division Risk

Task Add division-level risk rollups from real backend
 data

Difficulty Medium

Est. Hours 8

Dependencies Behavior model

Files Likely Affected Backend summary APIs, admin dashboard

Tags CR

Feature Badge System

Task Normalize badge definitions and frontend display
from one backend source

Difficulty Medium

Est. Hours 6

Dependencies Current badges config

Files Likely Affected backend/badges.json, reporting widget

Tags GFI

Feature Anti-Gaming

Task Add reporting quality checks: duplicate rate,
false positive rate, spam limits

Difficulty Medium

Est. Hours 10

Dependencies Reporting verdict model

Files Likely Affected Backend reports logic

Tags Post-MVP

Feature Training Eligibility

Task Fix event naming mismatch: eligibility checks
phishing_click, recorder uses clicked_link

Difficulty Easy

Est. Hours 2

Dependencies None

Files Likely Affected backend/routes/events.py

Tags CR, GFI

Epic 7: Threat Reporting Pipeline

Feature	Report Normalization
Task	Define canonical report object for URL/file reports
Difficulty	Medium
Est. Hours	5
Dependencies	Existing /api/reports
Files Likely Affected	backend/gamification_routes.py, docs
Tags	CR

Feature	Enrichment Storage
Task	Store VT/urlscan raw results and normalized verdicts in structured tables
Difficulty	Medium
Est. Hours	10
Dependencies	Unified DB
Files Likely Affected	backend/database.py, n8n workflow
Tags	CR

Feature	Report-to-Incident Link
Task	Link threat reports to incident tickets when severity crosses threshold
Difficulty	Medium
Est. Hours	8
Dependencies	Incident case model
Files Likely Affected	Backend report/incidents logic
Tags	CR

Feature	Reporter Feedback
Task	Return clear feedback message to Telegram: clean/suspicious/malicious + ticket ID

Difficulty	Medium
Est. Hours	6
Dependencies	n8n contract
Files Likely Affected	n8n workflow JSON, backend response shape
Tags	CR

Feature	File Reporting
Task	Add safe file metadata handling: hash, filename, size, MIME type, no unsafe execution
Difficulty	Medium
Est. Hours	8
Dependencies	Report normalization
Files Likely Affected	Backend reports, n8n workflow
Tags	CR

Feature	False Positive Workflow
Task	Allow SOC to mark report verdict as false positive and adjust gamification
Difficulty	Medium
Est. Hours	10
Dependencies	Case management
Files Likely Affected	Backend reports/incidents, admin UI
Tags	Post-MVP

Epic 8: GoPhish Campaign Management

Feature	Campaign Governance
Task	Add campaign draft/approval state before launch
Difficulty	Hard
Est. Hours	14
Dependencies	RBAC

Files Likely Affected Backend campaign table/routes, admin UI

Tags Post-MVP

Feature Target Segmentation

Task Add division/user filters before syncing GoPhish
target group

Difficulty Medium

Est. Hours 8

Dependencies User data model

Files Likely Affected backend/routes/admin_api.py, admin UI

Tags CR

Feature Campaign Results Sync

Task Periodically ingest GoPhish campaign results into
backend events

Difficulty Hard

Est. Hours 16

Dependencies GoPhish API

Files Likely Affected backend/gophish_client.py, backend scheduled job

Tags CR

Feature Campaign Templates

Task Display template/page/profile validation before
launch

Difficulty Medium

Est. Hours 6

Dependencies GoPhish resources API

Files Likely Affected dashboard/src/app/admin/page.tsx

Tags GFI

Feature Safety Controls

Task	Add frequency caps and quiet hours for simulations
Difficulty	Medium
Est. Hours	8
Dependencies	Campaign model
Files Likely Affected	Backend campaign launch logic
Tags	Post-MVP

Epic 9: Compliance & Executive Reporting

Feature	Compliance Model
Task	Map platform metrics to ISO 27001/NIST CSF/UU PDP style controls
Difficulty	Medium
Est. Hours	8
Dependencies	Real metrics
Files Likely Affected	backend/database.py, docs
Tags	CR

Feature	Executive Dashboard
Task	Add CISO summary cards: risk trend, reporting rate, click rate, MTTR, ROI
Difficulty	Medium
Est. Hours	10
Dependencies	Unified DB
Files Likely Affected	Admin dashboard, backend summary API
Tags	CR

Feature	Historical Trends
Task	Store daily snapshots for risk/compliance trend charts
Difficulty	Medium

Est. Hours 10

Dependencies Unified DB

Files Likely Affected Backend snapshot table/job

Tags Post-MVP

Feature Export Reports

Task CSV export for leaderboard, incidents, compliance
summary

Difficulty Easy

Est. Hours 6

Dependencies Backend APIs

Files Likely Affected Backend routes, admin UI

Tags GFI

Feature PDF Report

Task Generate monthly executive PDF report

Difficulty Medium

Est. Hours 12

Dependencies Export data

Files Likely Affected Backend report generation

Tags Post-MVP

Epic 10: Enterprise Integrations

Feature Identity Sync

Task Add placeholder interface for Entra ID/Google
Workspace/LDAP sync

Difficulty Medium

Est. Hours 8

Dependencies User model

Files Likely Affected Backend integration module

Tags Post-MVP

Feature SIEM Export

Task Add generic webhook/SIEM event export for
 incidents and reports

Difficulty Medium

Est. Hours 10

Dependencies Audit/event model

Files Likely Affected Backend integration module

Tags CR

Feature Ticketing Export

Task Add generic Jira/ServiceNow-style ticket webhook
 adapter

Difficulty Medium

Est. Hours 8

Dependencies Case management

Files Likely Affected Backend integration module

Tags Post-MVP

Feature Teams/Slack Alerts

Task Add notification adapter interface beyond Telegram

Difficulty Medium

Est. Hours 8

Dependencies Alert abstraction

Files Likely Affected Backend/n8n workflow

Tags Post-MVP

Feature Integration Settings

Task Add admin UI for integration status and test
 connection

Difficulty	Medium
Est. Hours	10
Dependencies	Integration adapters
Files Likely Affected	Admin dashboard
Tags	Post-MVP

Epic 11: Observability, Testing & Quality

Feature	Backend Tests
Task	Add unit tests for scoring, badge thresholds, dedupe, quiz streak
Difficulty	Medium
Est. Hours	12
Dependencies	None
Files Likely Affected	backend/test_gamification_manual.py, new tests
Tags	CR

Feature	API Tests
Task	Add endpoint tests for auth, events, reports, incidents
Difficulty	Medium
Est. Hours	16
Dependencies	Test setup
Files Likely Affected	Backend tests
Tags	CR

Feature	Frontend Checks
Task	Ensure npm run lint passes and fix TypeScript issues
Difficulty	Easy
Est. Hours	6

Dependencies None

Files Likely Affected dashboard

Tags CR, GFI

Feature E2E Demo Test

Task Add Playwright happy-path test for auth → employee
 dashboard → admin dashboard

Difficulty Hard

Est. Hours 18

Dependencies Stable auth/demo seed

Files Likely Affected Dashboard tests

Tags Post-MVP

Feature Structured Logging

Task Add JSON logs for backend requests, reports,
 incidents, auth failures

Difficulty Medium

Est. Hours 8

Dependencies None

Files Likely Affected backend/app.py, route modules

Tags CR

Feature Health Checks

Task Add readiness checks for Flask DB, GoPhish, n8n,
 dashboard

Difficulty Medium

Est. Hours 8

Dependencies Docker setup

Files Likely Affected docker-compose.yml, backend health route

Tags CR

Epic 12: Deployment & Production Readiness

Feature	Database Upgrade
Task	Plan migration path from SQLite to PostgreSQL
Difficulty	Hard
Est. Hours	12
Dependencies	Unified data model
Files Likely Affected	Backend DB layer, Docker Compose
Tags	Post-MVP

Feature	Migration Tooling
Task	Add Alembic or equivalent migration system
Difficulty	Hard
Est. Hours	12
Dependencies	DB upgrade decision
Files Likely Affected	Backend DB structure
Tags	Post-MVP

Feature	Secrets Management
Task	Remove fallback secrets and document required env vars
Difficulty	Easy
Est. Hours	4
Dependencies	None
Files Likely Affected	docker-compose.yml, env examples
Tags	CR, GFI

Feature	Config Validation
Task	Fail fast with clear messages for missing critical env vars
Difficulty	Easy

Est. Hours 4

Dependencies None

Files Likely Affected backend/app.py, Next config routes

Tags CR, GFI

Feature Container Hardening

Task Add non-root containers, health checks, restart
policy review

Difficulty Medium

Est. Hours 10

Dependencies Docker familiarity

Files Likely Affected Dockerfiles, Compose

Tags Post-MVP

Feature Backup/Restore

Task Add backup/restore process for persistent DB
volumes

Difficulty Medium

Est. Hours 8

Dependencies Persistence strategy

Files Likely Affected Docs/scripts

Tags Post-MVP

Recommended Competition MVP Scope

Build these first:

1. Epic 1: Demo integrity and polish.
2. Epic 2: Unified data layer for visible dashboard data.
3. Epic 3: Auth/RBAC minimum viable hardening.
4. Epic 4: Webhook/API security basics.

5. Epic 5: Incident case management basics.
6. Epic 6: Explainable score changes.
7. Epic 7: Report-to-incident linkage.
8. Epic 11: Tests and structured logging.

Do Not Prioritize Before Competition

- Full LMS.
- Many additional games.
- Custom SIEM.
- Custom phishing engine.
- Multi-tenant SaaS.
- Browser extension enforcement.
- PDF-heavy reporting system.
- Deep Entra/LDAP production sync.
- Complex AI assistant/chatbot.

The highest-leverage competition move is to make the existing story real, secure, explainable, and repeatable.

Current Implementation Summary

Next.js currently has a split personality:

- Some API routes are real backend proxies to Flask.
- Some API routes are local demo endpoints backed by `dashboard/src/lib/store.ts`.
- Some routes are hardcoded mock data without `dataStore`, but still not backend-backed.

For Epic 2, the main issue is that the SOC dashboard is not fully driven by real Flask/SQLite state. The employee reporting/gamification backend is more real than the SOC overview.

Routes Using In-Memory datastore

Route GET/POST /api/incident

File dashboard/src/app/api/incident/route.ts

Current State Stores incidents in Next.js memory and seeds demo incidents

Why It Exists Built as a presentation-layer webhook buffer for demo SOC incidents. Lets UI look alive without depending on Flask.

Route GET/POST /api/behavior

File dashboard/src/app/api/behavior/route.ts

Current State Stores behavior scores in memory and seeds demo users

Why It Exists Built to feed employee dashboard, admin risk charts, leaderboard-like views, and score cards before backend scoring was complete.

Route GET/POST /api/cache

File dashboard/src/app/api/cache/route.ts

Current State Stores threat-cache entries in memory

Why It Exists Demo endpoint for blocked URLs, threat score, source, and action shown in SOC dashboard.

Route GET/POST /api/policy

File dashboard/src/app/api/policy/route.ts

Current State Stores adaptive policy decisions in memory

Why It Exists Demo endpoint for the “adaptive enforcement” concept: threat score + behavior score → allow/warn/block/notify.

Route GET/POST /api/summary
File dashboard/src/app/api/summary/route.ts
Current State Stores AI summaries in memory
Why It Exists Demo endpoint for SOC-style generated summaries and recommendations.

Route GET/POST /api/auth/magic-link
File dashboard/src/app/api/auth/magic-link/route.ts
Current State Creates and validates magic-link tokens in memory; also registers token in Flask
Why It Exists Exists to let Telegram/n8n generate dashboard links quickly. It partially persists to Flask, but validation still depends on the Next.js in-memory token map.

Routes Not Using datastore But Still Mock/In-Memory Equivalent

Route GET /api/admin/login-history
File dashboard/src/app/api/admin/login-history/route.ts
Current State Hardcoded login events
Why It Exists Demo data for suspicious login/risk behavior panel.

Route GET /api/behavior/history
File dashboard/src/app/api/behavior/history/route.ts
Current State Hardcoded per-user login history
Why It Exists Demo data for identity/device/location risk history.

Priority Ranking

1. /api/incident

Highest priority. Incidents are central to the SOC story, and Flask

already has a real incidents table plus /api/incidents. This is the fastest path to replacing fake SOC data with real persisted state.

2. /api/behavior

Very high priority. It powers employee dashboard, admin overview, risk charts, and perceived product intelligence. Backend already has user_history, points, badges, leaderboard, and events, but the response shape needs to be adapted.

3. /api/auth/magic-link

High priority, but belongs partly to Epic 3. It uses in-memory token validation, which means auth can break after Next.js restart. Since Flask already has dashboard_tokens, this should eventually become Flask-owned.

4. /api/cache

Medium-high priority. Threat cache supports the “adaptive gateway / blocked URL” story. There is no real backend table yet, so this needs schema work.

5. /api/policy

Medium priority. Important differentiator, but currently conceptual. Should be persisted after behavior and threat cache are real.

6. /api/summary

Medium-low priority. Useful for SOC polish, but not foundational. Can be backend-backed after incidents/threats exist.

7. /api/admin/login-history

Lower priority for Epic 2. It is demo-only and not connected to current core workflows.

8. /api/behavior/history

Lower priority. Nice identity-risk demo, but not essential to the phishing/reporting competition story.

Recommended First Task

Start with migrating /api/incident from Next.js memory to Flask/SQLite.

Why this first:

- Backend persistence already exists in incidents.
- n8n/reporting flows already create real incident tickets through Flask.
- The SOC dashboard becomes immediately more credible.
- It reduces demo drift: reports and tickets shown in the UI can come from real backend state.
- It is smaller than behavior-score migration and less risky than auth migration.

First implementation task should be:

- > Replace dashboard/src/app/api/incident/route.ts with a Flask proxy/
- > adapter that reads from GET /api/incidents, maps Flask incident fields
- > to the dashboard's expected Incident shape, and computes dashboard stats
- > from persisted backend data.

I would not start by creating new schemas for cache/policy/summary yet.

Lock the real incident loop first, then move behavior scoring next.

Current Flask Schema

Existing implementation is in backend/routes/incidents.py and backed by backend/database.py.

Flask exposes three incident endpoints:

POST /api/incidents

GET /api/incidents

PATCH /api/incidents/<ticket_id>

GET /api/incidents accepts optional query params:

source_type=simulation | real_world_report

status=open | closed

Response shape:

```
{
  "incidents": [
    {
      "ticket_id": "INC-ABC12345",
      "source_type": "real_world_report",
      "reported_url": "https://example.test",
      "divisi": "Network Engineering",
      "severity": "high",
      "vt_verdict": "8/90 engines malicious",
      "urlscan_verdict": "malicious",
      "screenshot_url": "https://...",
      "checklist": "...",
      "file_hash": "sha256...",
    }
  ]
}
```



```
"original_filename": "invoice.pdf",  
"status": "open",  
"created_at": "2026-07-09 12:34:56",  
"closed_at": null  
}  
],  
"count": 1  
}
```

Fields returned per incident are all columns from the incidents table:

- ticket_id
- source_type
- reported_url
- divisi
- severity
- vt_verdict
- urlscan_verdict
- screenshot_url
- checklist
- file_hash
- original_filename
- status
- created_at
- closed_at

POST /api/incidents request schema:

```
{  
  "source_type": "simulation",  
  "divisi": "Network Engineering",
```

```
"severity": "low",
"reported_url": "https://example.test",
"vt_verdict": "clean",
"urlscan_verdict": "clean",
"screenshot_url": "https://...",
"checklist": "...",
"file_hash": "sha256...",
"original_filename": "invoice.pdf",
"reporter_chat_id": "123456789"
}
```

Required:

- source_type
- divisi

Validated:

- source_type: simulation, real_world_report
- severity: low, medium, high

Response on create:

```
{
"message": "Ticket incident berhasil dibuat",
"ticket_id": "INC-ABC12345",
"source_type": "real_world_report",
"severity": "high",
"points_awarded": 15,
"reporter": {
"email": "...",
```

```
"points": 115,  
  "badge": "Guardian"  
}  
}
```

points_awarded and reporter only appear for real_world_report incidents with medium or high severity when reporter_chat_id maps to a known employee.

PATCH /api/incidents/<ticket_id> request schema:

```
{  
  "status": "closed"  
}
```

Allowed status:

- open
- closed

Response:

```
{  
  "message": "Ticket INC-ABC12345 status diupdate ke closed"  
}
```

Current Dashboard Schema

The dashboard expects this interface in dashboard/src/app/admin/page.tsx:

```
interface Incident {
```

```
id: string;
timestamp: string;
type: string;
severity: string;
source: string;
target: string;
description: string;
status: string;
}
```

The existing Next.js route `dashboard/src/app/api/incident/route.ts` returns:

```
{
  "incidents": [
    {
      "id": "INC-001",
      "timestamp": "2026-07-09T12:34:56.000Z",
      "type": "phishing_report",
      "severity": "medium",
      "source": "Telegram Bot Report",
      "target": "sari@netops-dummy.local",
      "description": "User reported suspicious email...",
      "status": "resolved",
      "assignee": "SOC-Team"
    }
  ],
  "stats": {
    "totalIncidents": 5,
    "openIncidents": 1,
    "criticalIncidents": 1,
  }
}
```

```
"blockedUrls": 3,  
"totalEmployees": 8,  
"avgBehaviorScore": 72  
}  
}
```

The admin page currently reads:

- inc.id
- inc.timestamp
- inc.type
- inc.severity
- inc.source
- inc.description
- inc.status
- stats.totalIncidents
- stats.openIncidents
- stats.blockedUrls
- stats.avgBehaviorScore

target is part of the interface but not visibly used in the recent incident block.

Field Mapping Table

Dashboard Field	Flask Field	Transformation
id	ticket_id	Direct rename
timestamp	created_at	Direct rename;

		optionally normalize to ISO string	
type	source_type, plus file/ url fields	Derive dashboard- friendly type	
severity	severity	Direct, but Flask only supports low/ medium/high; dashboard also supports critical	
source	source_type	Convert enum to display label	
target	reported_url, original_filename, file_hash, divisi	Prefer URL, then filename, then hash, then division	
description	multiple fields	Compose readable description	
status	status	Map open/closed to dashboard status vocabulary if	

		needed	
assignee	none	Missing; omit or default to SOC- Team if interface is extended	
stats.totalIncidents	count of Flask incidents	Compute in Next.js adapter	
stats.openIncidents	incidents where status === "open"	Compute in adapter	
stats.criticalIncidents	none directly	Compute as severity high for now, or 0 until critical exists	
stats.blockedUrls	none in Flask incidents	Cannot derive accurately from incidents	
stats.totalEmployees	none from incident endpoint	Needs behavior/ user endpoint	
stats.avgBehaviorScore	none from incident endpoint	Needs behavior/ user endpoint	

Required Transformations

Recommended adapter behavior in dashboard/src/app/api/incident/route.ts:

1. Fetch Flask:

GET \${API_URL}/api/incidents

Authorization: Bearer \${SECRET_KEY}

2. Convert each Flask incident into dashboard incident shape:

id = ticket_id

timestamp = created_at

severity = severity

status = status === 'closed' ? 'resolved' : 'open'

source = source_type === 'real_world_report'

 ? 'Telegram Bot Report'

 : 'GoPhish Simulation'

type = deriveIncidentType(row)

target = reported_url || original_filename || file_hash || divisi ||

'Unknown'

description = buildIncidentDescription(row)

3. Suggested type derivation:

Flask Condition	Dashboard type
source_type === "simulation"	phishing_click
source_type === "real_world_report"	malware_detected
and original_filename or file_hash	

exists

source_type === "real_world_report" phishing_report or suspicious_url
and reported_url exists

fallback suspicious_url

For demo clarity, I recommend:

- URL reports: phishing_report
- File/hash reports: malware_detected
- Simulation: phishing_click

4. Suggested description composition:

For simulation:

Simulation incident for {divisi}

For URL report:

Reported URL {reported_url} from {divisi}. VT: {vt_verdict || "n/a"},
urlscan: {urlscan_verdict || "n/a"}.

For file report:

Reported file {original_filename || "unknown file"} from {divisi}. Hash:
{file_hash || "n/a"}.

5. Compute stats locally in the adapter:

totalIncidents = incidents.length

openIncidents = incidents.filter(status open).length

criticalIncidents = incidents.filter(severity high or critical).length

For now:

blockedUrls = 0

totalEmployees = 0

avgBehaviorScore = 0

But that would make overview cards regress. Better short-term adapter should preserve those stats from other real endpoints if cheap:

- avgBehaviorScore can eventually come from /api/behavior.
- blockedUrls can eventually come from /api/cache.
- For Task 2.1, either leave as 0 or keep a conservative fallback until cache/behavior are migrated.

Missing Fields

Missing from Flask compared with dashboard:

- id by name, but available as ticket_id.
- timestamp by name, but available as created_at.
- type is not stored directly.
- source display label is not stored directly.
- target is not stored directly.
- description is not stored directly.
- assignee is not available.
- critical severity is not supported by Flask; only low, medium, high.
- investigating, resolved, escalated statuses are not supported by Flask;

only open, closed.

Missing from dashboard compared with Flask:

- source_type
- reported_url
- divisi
- vt_verdict
- urlscan_verdict
- screenshot_url
- checklist
- file_hash
- original_filename
- closed_at

These are valuable SOC fields but currently hidden by the dashboard interface.

Potential Breaking Changes

- If the adapter returns raw Flask incidents directly, the dashboard will break because it expects id, timestamp, type, source, target, and description.
- If status remains closed, current UI will display it, but any badge styling expecting resolved may not match.
- If severity remains limited to high, the existing critical incident count and critical chart slices may drop.
- If created_at is not parseable by new Date(), timeAgo() may display

incorrect values. SQLite timestamp strings usually work, but ISO normalization is safer.

- If stats are not returned, overview cards will show —.
- If blockedUrls and avgBehaviorScore become 0, the dashboard may look less impressive even though incidents are real.
- POST compatibility matters: existing external services may POST to Next.js /api/incident, while n8n may POST to Flask /api/incidents. The adapter should either proxy POST to Flask or keep POST support during transition.

Recommended Adapter Layer

Minimize frontend changes by keeping /api/incident response exactly as the dashboard already expects:

```
{
  "incidents": [
    {
      "id": "INC-ABC12345",
      "timestamp": "2026-07-09T12:34:56.000Z",
      "type": "phishing_report",
      "severity": "high",
      "source": "Telegram Bot Report",
      "target": "https://example.test",
      "description": "Reported URL https://example.test from Network Engineering. VT: 8/90 engines malicious, urlscan: malicious.",
      "status": "open"
    }
  ],
}
```

```
"stats": {  
  "totalIncidents": 1,  
  "openIncidents": 1,  
  "criticalIncidents": 1,  
  "blockedUrls": 0,  
  "totalEmployees": 0,  
  "avgBehaviorScore": 0  
}  
}
```

This lets us migrate the data source without touching `dashboard/src/app/admin/page.tsx`. Later, we can enrich the frontend to show Flask-native fields like VT verdict, urlscan verdict, file hash, screenshot, and checklist.

Migration Checklist By File

`backend/app.py`

Current server-to-server auth:

- Reads SECRET_KEY from env.
- Uses SECRET_KEY both as Flask app.secret_key and as the bearer token for protected API routes.
- Global guard checks:

Authorization: Bearer <SECRET_KEY>

Must migrate:

- Add `SERVICE_API_KEY = os.environ.get("SERVICE_API_KEY")`.
- Keep `SECRET_KEY` only for Flask session signing.
- Use `SERVICE_API_KEY` for bearer authorization in `require_admin_for_protected_routes`.
- Fail fast if `SERVICE_API_KEY` is missing.
- Update comments to stop calling `SECRET_KEY` the server-to-server shared secret.

Decision:

- Must migrate.
- This is the central enforcement point for protected Flask APIs.

`docker-compose.yml`

Current usage:

- `n8n` receives `SECRET_KEY=${SECRET_KEY}`.
- `flask_api` receives `SECRET_KEY=${SECRET_KEY}`.
- `dashboard` does not receive `SECRET_KEY`.
- `dashboard` also does not receive any service auth key.

Must migrate:

- Add `SERVICE_API_KEY=${SERVICE_API_KEY}` to `flask_api`.
- Add `SERVICE_API_KEY=${SERVICE_API_KEY}` to `dashboard`.
- Add `SERVICE_API_KEY=${SERVICE_API_KEY}` to `n8n` if `n8n` workflows will continue calling protected Flask endpoints.

- Keep SECRET_KEY=\${SECRET_KEY} in flask_api for Flask sessions.
- Keep or remove SECRET_KEY from n8n depending on workflow migration timing; after migration, n8n should not need it for Flask API auth.

Decision:

- Must migrate for flask_api and dashboard.
- Must migrate for n8n if Flow B still posts to protected endpoints like /api/reports.

env (1).example

Current usage found:

```
# shared secret buat server-to-server auth via "Authorization: Bearer
<SECRET_KEY>"
SECRET_KEY=...
```

Must migrate:

- Change the comment: SECRET_KEY is Flask/session signing secret.
- Add:

```
SERVICE_API_KEY=...
```

- Document that server-to-server calls must use:

```
Authorization: Bearer <SERVICE_API_KEY>
```

Decision:

- Must migrate.
- This prevents future operators from reusing Flask session secret as an API credential.

— — —

backend/gamification_routes.py

Current usage:

- Comments say POST /api/reports is protected by:

Authorization: Bearer <SECRET_KEY>

- No direct auth logic in this file; it relies on backend/app.py.

Must migrate:

- Update comments to say:

Authorization: Bearer <SERVICE_API_KEY>

- No behavior change needed inside this file if app.py guard is updated.

Decision:

- Must migrate comments/documentation.
- Runtime migration happens through app.py.

n8n-workflows/Flow B — Threat Reporting (Fully Configured).json

Current usage:

```
"name": "Authorization",  
"value": "=Bearer {{ $env.SECRET_KEY }}"
```

This is likely used when n8n calls protected Flask endpoints, especially /api/reports.

Also found API-Key headers, but those appear to be third-party service keys, likely VirusTotal/urlscan style, not internal service auth.

Must migrate:

- Change internal Flask auth header to:

```
"value": "=Bearer {{ $env.SERVICE_API_KEY }}"
```

- Keep third-party API-Key headers unchanged unless they are confirmed internal.

- Ensure Docker passes SERVICE_API_KEY into n8n.

Decision:

- Must migrate.
- Otherwise protected Flow B backend calls will fail after Flask switches

away from SECRET_KEY.

dashboard/src/app/api/admin/compliance-summary/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-  
production';
```

Authorization: Bearer \${secretKey}

Must migrate:

- Use process.env.SERVICE_API_KEY.
- Remove fallback credential.
- Fail clearly if SERVICE_API_KEY is missing.
- Ideally share this through the new safeFetch/backend helper.

Decision:

- Must migrate.
- This route calls protected Flask /api/compliance-summary.

dashboard/src/app/api/admin/emails/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-
```

production';

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.
- This proxies to Flask /api/emails.

Decision:

- Must migrate.
- Even though Flask currently lists events.list_emails as public, this route should not rely on public access long-term.

dashboard/src/app/api/admin/leaderboard/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-production';
```

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.

Decision:

- Must migrate.
- Proxies protected Flask /api/leaderboard.

dashboard/src/app/api/admin/gophish/campaigns/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-production';
```

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.

Decision:

- Must migrate.
- Proxies protected Flask GoPhish endpoint.

dashboard/src/app/api/admin/gophish/resources/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-
```

production';

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.

Decision:

- Must migrate.

dashboard/src/app/api/admin/gophish/sync/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-  
production';
```

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.

Decision:

- Must migrate.

dashboard/src/app/api/admin/gophish/launch/route.ts

Current usage:

```
const secretKey = process.env.SECRET_KEY || 'dev-fallback-key-change-in-production';
```

Authorization: Bearer \${secretKey}

Must migrate:

- Use SERVICE_API_KEY.
- Remove fallback.

Decision:

- Must migrate.

dashboard/src/app/api/incident/route.ts

Current usage:

- No server-to-server auth yet.
- Uses in-memory datastore.

Upcoming Task 2.1 requirement:

- When migrated to Flask-backed adapter, it must call protected Flask /

api/incidents.

- It should use:

Authorization: Bearer \${process.env.SERVICE_API_KEY}

Must migrate:

- Add service auth when replacing in-memory route.
- Do not use SECRET_KEY.
- Do not add fallback credential.

Decision:

- Must use SERVICE_API_KEY during Task 2.1.

dashboard/src/app/api/auth/magic-link/route.ts

Current behavior:

- Calls Flask:

POST /api/auth/register-token

- Does not send Authorization.

Important finding:

- auth.register_token is not listed in PUBLIC_ROUTES in backend/app.py.

- That means this call should be protected by the global Flask guard.
- Current code catches and logs failures, so token registration may silently fail.

Must migrate:

- Add Authorization: Bearer <SERVICE_API_KEY> to this backend call.
- Use safe fetch/error handling.
- Stop silently swallowing backend token registration failure if backend token validation depends on it.

Decision:

- Must migrate.
- This is not part of Incident Task 2.1, but it is a real existing auth bug.

backend/gophish_client.py

Current usage:

'Authorization': f'Bearer {GOPHISH_API_KEY}'

This is not internal server-to-server auth. It authenticates Flask to GoPhish.

Must migrate:

- No migration to SERVICE_API_KEY.

- Keep GOPHISH_API_KEY.

Decision:

- Do not migrate.
- Separate third-party/product API credential.

backend/debug_auth.log

Current usage:

- Contains logged SECRET_KEY and bearer token values.

Must migrate:

- Remove from repository if possible.
- Ensure future logs never print secrets.
- Add to .gitignore if not already ignored.

Decision:

- Security cleanup required, but not a SERVICE_API_KEY migration target.
- This is sensitive credential leakage.

Third-party API-Key Headers In n8n Workflow

Found in:

- n8n-workflows/Flow B — Threat Reporting (Fully Configured).json

Current usage:

"name": "API-Key"

Likely for VirusTotal/urlscan.

Must migrate:

- Do not migrate unless inspection proves the target is internal Flask.
- Third-party API keys should remain VT_API_KEY / URLSCAN_API_KEY.

Decision:

- Do not migrate by default.

— — —

Routes That Call Flask But Do Not Need SERVICE_API_KEY Yet

These call public or employee-token-protected Flask endpoints:

- dashboard/src/app/api/event/route.ts
- dashboard/src/app/api/user-eligibility/route.ts
- dashboard/src/app/api/user-activity/route.ts
- dashboard/src/app/api/employee/[id]/reports-summary/route.ts
- dashboard/src/app/api/auth/admin-login/route.ts

Migration decision:

- Do not blindly add SERVICE_API_KEY.
- These are either public by design or validate employee/admin credentials through request body/query token.
- Adding service auth here could blur user identity boundaries.

— — —

Recommended Migration Order

1. Add SERVICE_API_KEY env documentation and Docker propagation.
2. Update Flask global guard to validate SERVICE_API_KEY, not SECRET_KEY.
3. Update all Next.js admin proxy routes to use SERVICE_API_KEY.
4. Update Task 2.1 /api/incident adapter to use SERVICE_API_KEY.
5. Update n8n Flow B Authorization header to use \$env.SERVICE_API_KEY.
6. Update comments in gamification_routes.py.
7. Fix auth/magic-link token registration to authenticate with SERVICE_API_KEY.
8. Remove/debug sanitize backend/debug_auth.log.

Critical Competition Readiness Items

- backend/app.py
- docker-compose.yml
- all dashboard/src/app/api/admin/** proxy routes using SECRET_KEY
- dashboard/src/app/api/incident/route.ts during Task 2.1
- n8n Flow B internal Authorization header
- env (1).example

These must be migrated together or the dashboard/Flow B will break once Flask stops accepting SECRET_KEY as a bearer token.

– Worked for 1m 08s

› Before implementing the SERVICE_API_KEY migration, create a migration impact report.

For every modified file, classify it as:

- Breaking Change
- Backward Compatible
- Requires Environment Update
- Requires Docker Restart
- Requires n8n Workflow Re-import

Output a migration checklist that can be followed during deployment.

Do not modify code yet.

- Migration Impact Report

Scope: migrate internal server-to-server auth from SECRET_KEY to SERVICE_API_KEY, while keeping SECRET_KEY for Flask session signing only.

File	Expected Change	Classification
backend/app.py	Add SERVICE_API_KEY	Breaking Change,

env requirement and Requires Environment
validate Update, Requires
Authorization: Bearer Docker Restart
<SERVICE_API_KEY>
instead of SECRET_KEY

docker-compose.yml Pass SERVICE_API_KEY Requires Environment
into flask_api, Update, Requires
dashboard, and n8n Docker Restart
services

env (1).example Add SERVICE_API_KEY; Backward Compatible,
clarify SECRET_KEY is Requires Environment
no longer service Update
auth

dashboard/src/app/ Use SERVICE_API_KEY Breaking Change if env
api/admin/compliance- for Flask bearer auth missing, Requires
summary/route.ts Environment Update,
Requires Docker
Restart

dashboard/src/app/ Use SERVICE_API_KEY Breaking Change if env
api/admin/emails/ for Flask bearer auth missing, Requires
route.ts Environment Update,
Requires Docker
Restart

dashboard/src/app/ Use SERVICE_API_KEY Breaking Change if env
api/admin/ for Flask bearer auth missing, Requires
leaderboard/route.ts Environment Update,

Requires Docker

Restart

dashboard/src/app/ api/admin/gophish/ campaigns/route.ts	Use SERVICE_API_KEY for Flask bearer auth missing, Requires Environment Update, Requires Docker Restart	Breaking Change if env
--	--	------------------------

dashboard/src/app/ api/admin/gophish/ resources/route.ts	Use SERVICE_API_KEY for Flask bearer auth missing, Requires Environment Update, Requires Docker Restart	Breaking Change if env
--	--	------------------------

dashboard/src/app/ api/admin/gophish/ sync/route.ts	Use SERVICE_API_KEY for Flask bearer auth missing, Requires Environment Update, Requires Docker Restart	Breaking Change if env
---	--	------------------------

dashboard/src/app/ api/admin/gophish/ launch/route.ts	Use SERVICE_API_KEY for Flask bearer auth missing, Requires Environment Update, Requires Docker Restart	Breaking Change if env
---	--	------------------------

dashboard/src/app/ api/auth/magic-link/ route.ts	Add SERVICE_API_KEY bearer auth when registering Flask dashboard token missing, Requires	Backward Compatible if env configured; Breaking Change if env
--	--	---

Environment Update,
Requires Docker
Restart

dashboard/src/app/ As part of Task 2.1, Breaking Change if env
api/incident/route.ts use SERVICE_API_KEY missing, Requires
when proxying to Environment Update,
Flask /api/incidents Requires Docker
Restart

[dashboard/src/lib/ New safeFetch helper Backward Compatible,
backend.ts] or reads SERVICE_API_KEY Requires Environment
similar new helper and handles backend Update if used by
failures protected routes

backend/ Update comments from Backward Compatible
gamification_routes.p SECRET_KEY to
y SERVICE_API_KEY

n8n-workflows/Flow B Change internal Flask Breaking Change if n8n
— Threat Reporting auth header from env not updated,
(Fully \$env.SECRET_KEY to Requires Environment
Configured).json \$env.SERVICE_API_KEY Update, Requires
Docker Restart,
Requires n8n Workflow
Re-import

backend/ Remove/sanitize Backward Compatible
debug_auth.log committed secret-
bearing log

Breaking Change Summary

This migration is breaking because once backend/app.py stops accepting SECRET_KEY as bearer auth, every internal caller still using:

Authorization: Bearer <SECRET_KEY>

will receive 401 Unauthorized.

Affected callers:

- Dashboard admin proxy routes
- Future dashboard incident adapter
- n8n Flow B protected backend calls
- Magic-link token registration if it remains protected

Environment Updates Required

Add this variable to the real .env used by Docker Compose:

SERVICE_API_KEY=<new-long-random-service-token>

Keep this existing variable:

SECRET_KEY=<flask-session-secret>

Recommended meanings after migration:

SECRET_KEY = Flask session/cookie signing secret only

SERVICE_API_KEY = internal server-to-server API bearer credential

Do not reuse the same value for both.

Docker Restart Requirements

After updating `.env` and `docker-compose.yml`, restart services that read these env vars:

```
docker compose up -d --build flask_api dashboard n8n
```

Minimum restart:

```
docker compose up -d --build flask_api dashboard
```

If n8n workflow auth is migrated, restart n8n too:

```
docker compose up -d --build n8n
```

n8n Workflow Re-import Requirements

Required if the JSON workflow file is changed and the running n8n instance still has the old workflow definition.

Steps:

1. Update workflow JSON to use:

```
Bearer {{ $env.SERVICE_API_KEY }}
```

2. Ensure `SERVICE_API_KEY` is available in the n8n container.

3. Restart n8n.

4. Re-import or update the workflow in the n8n UI.

5. Toggle workflow inactive → active so webhook/runtime config refreshes.

Deployment Checklist

1. Generate a new long random service token.

SERVICE_API_KEY=<at least 32 bytes random>

2. Add SERVICE_API_KEY to local .env.

3. Update docker-compose.yml:

- flask_api.environment
- dashboard.environment
- n8n.environment

4. Update backend auth guard in backend/app.py:

- Require SERVICE_API_KEY
- Validate bearer token against SERVICE_API_KEY
- Keep SECRET_KEY for Flask session only

5. Update all dashboard protected Flask proxy routes to use
SERVICE_API_KEY.

6. Update /api/auth/magic-link token registration request to use
SERVICE_API_KEY.

7. Implement Task 2.1 incident adapter using SERVICE_API_KEY.

8. Update n8n Flow B Authorization header to \$env.SERVICE_API_KEY.

9. Rebuild/restart services:

`docker compose up -d --build flask_api dashboard n8n`

10. Re-import or refresh n8n workflow if using the JSON workflow file.

11. Smoke test:

- GET /health
- Admin login
- Admin compliance summary
- Admin leaderboard
- GoPhish resources/campaigns
- Magic-link dashboard token flow
- GET /api/incident from Next.js
- n8n Flow B report submission to Flask /api/reports

12. Verify no caller still uses Authorization: Bearer \${SECRET_KEY}.

13. Remove/sanitize backend/debug_auth.log because it contains secret material.

Safe Rollback Plan

If deployment fails:

1. Temporarily restore Flask guard to accept both SERVICE_API_KEY and SECRET_KEY.
2. Restart flask_api.
3. Migrate callers one by one.
4. Remove SECRET_KEY bearer fallback after all callers are confirmed migrated.

For competition readiness, I recommend avoiding dual-token fallback unless

needed for emergency rollback. It weakens the purpose of the migration.

› Implement Pull Request #1: Internal Service Authentication Migration.

Context:

We are preparing this repository for a cybersecurity competition.

This PR is ONLY about migrating internal service-to-service authentication from SECRET_KEY to SERVICE_API_KEY.

Do NOT work on Incident API migration yet.

Do NOT refactor unrelated code.

Objectives

1. Introduce SERVICE_API_KEY as the dedicated internal service credential.
2. Keep SECRET_KEY exclusively for Flask session/cookie signing.
3. Update every affected internal service caller.
4. Preserve existing functionality.
5. Keep the implementation minimal.

Implementation Scope

Backend

- backend/app.py
 - Read SERVICE_API_KEY from environment.
 - Fail fast if SERVICE_API_KEY is missing.
 - Validate Authorization: Bearer <SERVICE_API_KEY>.
 - Keep SECRET_KEY only for Flask session signing.

Infrastructure

- docker-compose.yml
- Pass SERVICE_API_KEY into:
 - flask_api
 - dashboard
 - n8n

Configuration

- .env.example
- Add SERVICE_API_KEY.
- Update comments to distinguish SECRET_KEY and SERVICE_API_KEY.

Dashboard

Update every protected Flask proxy route that currently sends:

Authorization: Bearer \${process.env.SECRET_KEY}

to instead use:

Authorization: Bearer \${process.env.SERVICE_API_KEY}

Remove insecure fallback credentials such as:

dev-fallback-key-change-in-production

Magic Link

- Update dashboard/src/app/api/auth/magic-link/route.ts
- Authenticate backend token registration using SERVICE_API_KEY.

n8n

- Update Flow B internal Flask Authorization header:

Bearer {{ \$env.SERVICE_API_KEY }}

Do not modify third-party API keys.

Documentation

- Update comments referring to SECRET_KEY as the service credential.

Requirements

- Keep changes as small as possible.
- Do not introduce unnecessary abstractions.
- Do not change frontend React components.
- Magic-link registration still works.
- GoPhish proxy routes still work.
- n8n Flow B authentication is updated.
- No internal route still uses SECRET_KEY as Bearer authentication.

Output

- I'll keep this PR narrowly scoped to the service credential migration:
backend guard, Docker/env docs, affected dashboard proxy routes, magic-link registration, n8n Flow B auth header, and secret-bearing debug log cleanup. No incident API migration.
- Running `Get-ChildItem -Force | Where-Object { $_.Name -like '*env*' }`
- Running `rg "process\.env\.SECRET_KEY |
| dev-fallback-key-change-in-production | Bearer \{\{ \
| $env\.SECRET_KEY \}\} | Authorization.*SECRET_KEY | <SECRET_KEY> |
| ... +2 lines`
- Ran `Get-Content -LiteralPath 'env (1).example'`

└ # .env.example " Human Firewall Lite

#

... +68 lines (ctrl + t to view transcript)

urlscan.io API key

-like '*env*' } this time

- Ran Get-ChildItem -Force | Where-Object { \$_.Name -like '*env*' }

└

... +7 lines (ctrl + t to view transcript)

✓ You approved codex to run rg "process\.env\SECRET_KEY|dev-fallback-key-change-in-production|Bearer \{... this time

"Authorization: Bearer <SECRET_KEY>"

... +21 lines (ctrl + t to view transcript)

```
dashboard\src\app\api\admin\gophish\sync\route.ts:6:  const secretKey
= process.env.SECRET_KEY || 'dev-fallback-key-change-in-production';
```

⚠ Heads up, you have less than 10% of your monthly limit left. Run /status for a breakdown.

-
- The repo has env (1).example rather than .env.example; I'll update the tracked example that exists. I found seven dashboard admin proxy routes using SECRET_KEY with an insecure fallback, plus backend comments and n8n Flow B references.

- Running Get-Content

```
| dashboard\src\app\api\admin\compliance-summary\route.ts;
| Get-Content dashboard\src\app\api\admin\emails\route.ts;
| ... +1 lines
```

- Running Get-Content dashboard\src\app\api\auth\magic-link\route.ts

- Running Get-Content

```
| dashboard\src\app\api\admin\gophish\campaigns\route.ts;
| Get-Content
| ... +3 lines
```

- Ran rg "SECRET_KEY|Authorization|Bearer" "n8n-workflows\Flow B — Threat

```
| Reporting (Fully Configured).json" -n -C 2
```

```
└─ 980-      "parameters": [
981-      {
... +2 lines (ctrl + t to view transcript)
984-      }
985-    ]
```

- Ran Get-Content dashboard\src\app\api\auth\magic-link\route.ts

```
└─ import { NextRequest, NextResponse } from 'next/server';
import { datastore } from '@lib/store';
... +77 lines (ctrl + t to view transcript)
});
}
```

```
└─ 980-      "parameters": [
981-      {
... +2 lines (ctrl + t to view transcript)
984-      }
985-    ]
```


- Ran Get-Content dashboard\src\app\api\admin\compliance-summary\route.ts;

```
| Get-Content dashboard\src\app\api\admin\emails\route.ts; Get-Content
| dashboard\src\app\api\admin\leaderboard\route.ts
```

```
└─ import { NextRequest, NextResponse } from 'next/server';
```

... +70 lines (ctrl + t to view transcript)

```
}
}
```

- Ran Get-Content dashboard\src\app\api\admin\gophish\campaigns\route.ts;

```
| Get-Content dashboard\src\app\api\admin\gophish\resources\route.ts;
```

```
| Get-Content dashboard\src\app\api\admin\gophish\sync\route.ts;
```

```
| ... +1 lines
```

```
└─ import { NextRequest, NextResponse } from 'next/server';
```

... +98 lines (ctrl + t to view transcript)

```
}
}
```

- I'm going to make the smallest direct changes: Flask will require SERVICE_API_KEY, dashboard proxy routes will read that env var without fallback, magic-link registration will include the bearer header, and Flow B will reference \$env.SERVICE_API_KEY.

- Edited backend\app.py (+7 -4)

cks on startup

```
11 ADMIN_PASSWORD = os.environ.get('ADMIN_PASSWORD')
```

```
12 SECRET_KEY = os.environ.get('SECRET_KEY')
```

```

13 +SERVICE_API_KEY = os.environ.get('SERVICE_API_KEY')
14 if not ADMIN_PASSWORD:
    :
17     raise RuntimeError("CRITICAL ERROR: Environment variable 'SECRET
        _KEY' is not set! Flask application refuses to start.")
18 +if not SERVICE_API_KEY:
19 +     raise RuntimeError("CRITICAL ERROR: Environment variable 'SERVIC
        E_API_KEY' is not set! Flask application refuses to start.")
20
    :
60
58 -     # 1. Cek shared secret header untuk server-to-server (Next.js / n8n)
61 +     # 1. Cek dedicated service API key untuk server-to-server (Next.js / n8n)
62     if auth_header and auth_header.startswith('Bearer '):
63         token = auth_header.split(' ')[1]
61 -         if token == SECRET_KEY:
64 +         if token == SERVICE_API_KEY:
65             return
    :
85     debug_mode = os.environ.get('FLASK_DEBUG', 'false').lower() == '
        true'
83 - app.run(host='0.0.0.0', port=5000, debug=debug_mode)
86 + app.run(host='0.0.0.0', port=5000, debug=debug_mode)

```

⚠ Heads up, you have less than 5% of your monthly limit left. Run `/status` for a breakdown.

- Edited docker-compose.yml (+12 -9)

```

23     - VT_API_KEY=${VT_API_KEY}

```

```

24 - - URLSCAN_API_KEY=${URLSCAN_API_KEY}
78 + - BOT_USERNAME=${BOT_USERNAME}
79 - GOPHISH_API_KEY=${GOPHISH_API_KEY}
    :
98 - NEXT_PUBLIC_API_URL=http://flask_api:5000
97 - - NEXT_PUBLIC_BOT_USERNAME=${BOT_USERNAME}
98 - - API_URL=http://flask_api:5000
99 + - NEXT_PUBLIC_BOT_USERNAME=${BOT_USERNAME}
100 + - API_URL=http://flask_api:5000
101 + - SERVICE_API_KEY=${SERVICE_API_KEY}
102 depends_on:
    :
114 hfl_network:
112 - driver: bridge
115 + driver: bridge

```

- Backend and Docker wiring are updated. Next I'll update the environment example and route callers; these are mechanical credential changes only.

- Ran `Get-Content -LiteralPath 'env (1).example' | Select-Object -Skip 12`

| -First 12

L

Password buat login admin dashboard (session-based auth)

... +11 lines (ctrl + t to view transcript)

- You've hit your usage limit. To continue using Codex and get access to

GPT-5.3-Codex, start a free trial of Plus today

(<https://chatgpt.com/explore/plus>), or try again at Aug 8th, 2026 12:28 AM.

› Implement Pull Request #1: Internal Service Authentication Migration.

Context:

We are preparing this repository for a cybersecurity competition.

This PR is ONLY about migrating internal service-to-service authentication from SECRET_KEY to SERVICE_API_KEY.

Do NOT work on Incident API migration yet.

Do NOT refactor unrelated code.

Objectives

1. Introduce SERVICE_API_KEY as the dedicated internal service credential.
2. Keep SECRET_KEY exclusively for Flask session/cookie signing.
3. Update every affected internal service caller.
4. Preserve existing functionality.
5. Keep the implementation minimal.

Implementation Scope

Backend

- backend/app.py
 - Read SERVICE_API_KEY from environment.
 - Fail fast if SERVICE_API_KEY is missing.
 - Validate Authorization: Bearer <SERVICE_API_KEY>.
 - Keep SECRET_KEY only for Flask session signing.

Infrastructure

- docker-compose.yml
- Pass SERVICE_API_KEY into:
 - flask_api
 - dashboard
 - n8n

Configuration

- .env.example
- Add SERVICE_API_KEY.
- Update comments to distinguish SECRET_KEY and SERVICE_API_KEY.

Dashboard

Update every protected Flask proxy route that currently sends:

Authorization: Bearer \${process.env.SECRET_KEY}

to instead use:

Authorization: Bearer \${process.env.SERVICE_API_KEY}

Remove insecure fallback credentials such as:

dev-fallback-key-change-in-production

Magic Link

- Update dashboard/src/app/api/auth/magic-link/route.ts
- Authenticate backend token registration using SERVICE_API_KEY.

n8n

- Update Flow B internal Flask Authorization header:

Bearer {{ \$env.SERVICE_API_KEY }}

Do not modify third-party API keys.

Documentation

- Update comments referring to SECRET_KEY as the service credential.

Requirements

- Keep changes as small as possible.
- Do not introduce unnecessary abstractions.
- Do not change frontend React components.

Testing

After implementation, verify:

- Flask starts with SERVICE_API_KEY.
- Dashboard admin proxy routes authenticate successfully.
- Magic-link registration still works.
- GoPhish proxy routes still work.
- n8n Flow B authentication is updated.
- No internal route still uses SECRET_KEY as Bearer authentication.

Output

After finishing, provide:

7. Pull Request summary.

Wait for my approval before starting Pull Request #2 (Incident API Migration).

- I'm updating only the auth surfaces in scope: Flask guard, Docker env wiring, the documented env example, dashboard proxy callers, the magic-link backend registration call, and the n8n Flow B internal bearer header. I'm leaving the incident API alone.

- Running Get-Content

```
| dashboard\src\app\api\admin\compliance-summary\route.ts
```

- Running Get-Content dashboard\src\app\api\admin\leaderboard\route.ts

- Running Get-Content

```
| dashboard\src\app\api\admin\gophish\campaigns\route.ts;
```

```
| Get-Content
```

```
| ... +3 lines
```

- Running Get-Content dashboard\src\app\api\admin\emails\route.ts

- Running Get-Content backend\gamification_routes.py

- Running Get-Content backend\debug_auth.log

- Ran Get-Content backend\app.py

```
L """
```

```
app.py â€” Flask API untuk Human Firewall Lite.
```

```
... +20 lines (ctrl + t to view transcript)
```

```
app = Flask(__name__)
```

- Ran Get-Content dashboard\src\app\api\admin\compliance-summary\route.ts

```
L import { NextRequest, NextResponse } from 'next/server';
```

... +22 lines (ctrl + t to view transcript)

```
}  
  
}
```

- Ran Get-Content dashboard\src\app\api\admin\leaderboard\route.ts

└─ import { NextRequest, NextResponse } from 'next/server';

... +22 lines (ctrl + t to view transcript)

```
}  
  
}
```

- Ran Get-Content dashboard\src\app\api\admin\gophish\campaigns\route.ts;

| Get-Content dashboard\src\app\api\admin\gophish\resources\route.ts;

| Get-Content dashboard\src\app\api\admin\gophish\sync\route.ts;

| ... +1 lines

└─ import { NextRequest, NextResponse } from 'next/server';

... +98 lines (ctrl + t to view transcript)

```
}  
  
}
```

└─ import { NextRequest, NextResponse } from 'next/server';

... +22 lines (ctrl + t to view transcript)

```
}  
  
}
```

- Ran Get-Content backend\debug_auth.log

└─ Path: /api/telegram/user

Auth Header: None

... +11 lines (ctrl + t to view transcript)

Match: True

- Ran Get-Content backend\app.py

L """

app.py â€” Flask API untuk Human Firewall Lite.

... +84 lines (ctrl + t to view transcript)

'true'

app.run(host='0.0.0.0', port=5000, debug=debug_mode)

- Ran Get-Content backend\gamification_routes.py

L """

gamification_routes.py â€” 3 endpoint gamifikasi sesuai api_contract.md.

... +189 lines (ctrl + t to view transcript)

return error_response(500, "INTERNAL_ERROR", f"Gagal memproses
quiz completion: {e}")

- You've hit your usage limit. To continue using Codex and get access to GPT-5.3-Codex, start a free trial of Plus today (<https://chatgpt.com/explore/plus>), or try again at Aug 8th, 2026 12:28 AM.

› udah semua?

- You've hit your usage limit. To continue using Codex and get access to GPT-5.3-Codex, start a free trial of Plus today (<https://chatgpt.com/explore/plus>), or try again at Aug 8th, 2026 12:28 AM.

› [main 2026-07-09T02

c

gpt-5.4-mini medium · C:\Human_Firewall